

Prompt

For my downloads folder write a script that do the following check for duplicates (same name or same contents), flag very large files, or group files by type.

“Show me your full plan first every file you would move, delete, or rename — and wait for my approval before touching anything.”

Logic

Here is the breakdown of the logic driving that script, split into how it **thinks** (the strategy) and how it **acts** (the execution).

1. The Discovery & Analysis Phase

Before the script decides to move or delete anything, it builds an internal map of your files using three core logical checks.

A. The Content Duplicate Logic (The Fingerprint)

Checking only file names misses duplicates that were renamed (e.g., presentation.pdf and presentation_final_v2_edit.pdf).

- **The Math:** The script reads the binary data of a file and creates a short, unique string called an MD5 hash. If two files have the exact same hash, their contents are 100% identical, no matter what they are named.
- **The Optimization:** Hashing a 4GB movie can take a long time and slow down your computer. To fix this, the script uses a "fast-hash" trick: if a file is larger than 100MB, it hashes only the first 5MB and the last 5MB. If a video has the exact same beginning and end bytes, it is almost certainly a duplicate.

B. The Name Duplicate Logic (The Cleanup)

Sometimes you download the exact same file twice, and your browser automatically saves it as document.pdf and document (1).pdf.

- The script keeps track of all file names. If it spots a name collision or standard browser-appended duplicates, it flags them.
- **The Priority Rule:** When deciding *which* duplicate to delete and which to keep, the script sorts them by age and name length. It intentionally keeps the **oldest file with the shortest name** (the original) and targets the newer copies for deletion.

C. The Large File Logic (The Space Saver)

This is a straightforward threshold filter. The script looks at the metadata (the file properties) of each item. If the size attribute is greater than \$500 \text{ MB}\$, it puts it on a separate warning list so you can see what is hogging your storage.

2. The Sorting Logic (Grouping by Type)

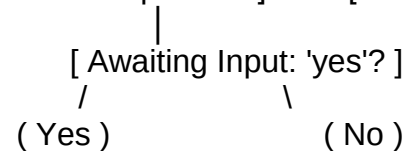
Once duplicates are filtered out, the script looks at the remaining unique files and checks their extensions (like .jpg or .pdf).

- It loops through a dictionary of predefined categories (Images, Documents, Videos, etc.).
- If a file's extension matches a category, it tags that file to be moved into that specific subfolder.
- If an extension doesn't match anything (like a random .xyz file), it assigns it to an Others folder so your main directory stays clean.

3. The Safety & "Dry Run" Logic

This is the most critical part of your prompt's requirements.

[Scan Files] —> [Group & Find Duplicates] —> [Print Detailed Plan]



/ \
[Execute Safely] [Abort & Do Nothing]

- **Memory Buffers:** The script does not move files as it finds them. Instead, it creates empty Python dictionaries (lists of key-value pairs) in your computer's short-term memory (actions_move and actions_delete). It populates these lists with instructions (e.g., *"File A should move to Folder B"*).
- **The Gatekeeper:** The script pauses and prints these memory lists to your screen. The actual file-handling commands (os.rename or path.rename) are trapped behind an if confirm.lower() == 'yes': condition. If you type anything else, the script ends, the short-term memory clears, and your hard drive remains entirely untouched.

4. The Execution Safeguards

If you do type yes, the script executes with two final safety nets:

1. **No Hard Deletion:** Instead of using a permanent delete command (which bypasses the recycling bin), the script moves "deleted" duplicates into a folder named _Duplicates right inside your Downloads. If it made a mistake, you can just drag them back out.
2. **Collision Prevention:** If it tries to move photo.jpg into the Images folder, but an older photo.jpg already exists there, the script checks for the conflict and automatically renames the new one to photo_1.jpg so it never accidentally overwrites your data.